

Fixing the "Cannot read properties of undefined (reading 'S')" Error

This error message usually indicates a misconfiguration in your React project rather than a bug in your own component code. In this case, the culprit was a **version mismatch and import map issues** in the project setup. There were a few specific problems identified:

- **Mismatched React versions:** Your `package.json` had `react` and `react-dom` on different major versions (React v18 vs ReactDOM v19). This mismatch can break React's internals.
- **Incorrect import map entries:** The HTML import map included a stray mapping for `react-dom/` pointing to ReactDOM 19, conflicting with React 18. It also had a truncated URL for the `@react-three/postprocessing` package, causing that module to load as `undefined`.

All these can trigger the kind of cryptic error (`reading 'S'`) you saw in the console. Below, we'll fix each issue step by step.

1. Align React and ReactDOM Versions

First, ensure **React and ReactDOM use the exact same version**. In your `package.json` dependencies, update ReactDOM to match React. For example, if React is `18.3.1`, set ReactDOM to `"18.3.1"` (not `^19.x.x`). Your dependencies section should look like this:

```
// package.json (after fixing versions)
"dependencies": {
  "react": "18.3.1",
  "react-dom": "18.3.1",
  "three": "0.161.0",
  "@react-three/fiber": "8.16.8",
  "@react-three/drei": "9.108.3",
  "postprocessing": "6.35.4",
  "@react-three/postprocessing": "2.16.2",
  "@google/genai": "^1.29.1"
}
```

After editing `package.json`, reinstall your dependencies to apply the change:

```
npm install
```

Having React and ReactDOM on different major versions (e.g. React 18 vs ReactDOM 19) will cause internal inconsistencies. In fact, React 19 introduced changes in the reconciler that libraries like React-Three-Fiber

(R3F) did not yet support, leading to undefined properties at runtime ¹. By using a matching pair (React 18.x with ReactDOM 18.x), you avoid those conflicts.

2. Fix Import Map Entries in index.html

Open your `index.html` and locate the `<script type="importmap">` block. We need to correct two things in the import map:

- **Remove the stray ReactDOM v19 mapping:** If there is an entry like `"react-dom/": "https://...react-dom^19.2.0/"`, remove it (and any similar React 19 entries). All React and ReactDOM imports should resolve to the same 18.3.1 version you set above.
- **Fix the `@react-three/postprocessing` URL:** Ensure the URL is complete (not truncated). In your case it was cut off as `.../@react-three/...g@2.16.2`, which is invalid. It should point to the full package name.

Use the following as a corrected import map (notice React and ReactDOM both point to 18.3.1, and the `@react-three/postprocessing` path is fixed):

```
<script type="importmap">
{
  "imports": {
    "react": "https://esm.sh/react@18.3.1",
    "react/": "https://esm.sh/react@18.3.1/",
    "react-dom": "https://esm.sh/react-dom@18.3.1",
    "react-dom/client": "https://esm.sh/react-dom@18.3.1/client?
deps=react@18.3.1",

    "three": "https://esm.sh/three@0.161.0",
    "three/": "https://esm.sh/three@0.161.0/",

    "@react-three/fiber": "https://esm.sh/@react-three/fiber@8.16.8?
deps=react@18.3.1,three@0.161.0",
    "@react-three/drei": "https://esm.sh/@react-three/drei@9.108.3?
deps=react@18.3.1,three@0.161.0",

    "postprocessing": "https://esm.sh/postprocessing@6.35.4?deps=three@0.161.0",
    "@react-three/postprocessing": "https://esm.sh/@react-three/
postprocessing@2.16.2?deps=react@18.3.1,three@0.161.0,postprocessing@6.35.4",

    "@google/genai": "https://aistudiocdn.com/@google/genai^1.29.1"
  }
}
</script>
```

Key changes: We removed any reference to ReactDOM 19 (the `"react-dom/"` mapping was dropped), and corrected the `@react-three/postprocessing` URL. This ensures all modules are pulled in with a

consistent, compatible version set. Now, the React Three Fiber packages will all use `three@0.161.0` and React 18, avoiding undefined imports.

3. Reinstall and Restart the Dev Server

After making the above changes:

- **Reinstall dependencies** (if you changed `package.json` in step 1) by running `npm install` again. This guarantees that `react-dom` v18.3.1 is in place instead of v19.
- **Restart your development server.** For example, if using Vite:

```
npm run dev
```

- **Clear the browser cache** (if necessary) and reload your app. Then check the console.

With React and ReactDOM versions aligned and the import map fixed, the error `Uncaught TypeError: Cannot read properties of undefined (reading 'S')` should disappear. Essentially, we've eliminated the version conflict that was causing ReactDOM 19 code to look for an internal property in React 18 that didn't exist. We also fixed the broken module import that could return `undefined`.

At this point, your 3D scene should load without that runtime error. If you do encounter a new error message, take note of its details and stack trace — it will provide clues for the next steps (feel free to ask again with that information!). In summary, keeping React/ReactDOM versions in sync and ensuring import paths are correct resolved the `'S' is undefined` issue ¹.

¹ React 19 support - "Cannot read properties of undefined (reading 'ReactCurrentOwner')" · Issue #3398 · pmndrs/react-three-fiber · GitHub

<https://github.com/pmndrs/react-three-fiber/issues/3398>